

Applying Pseudoinverse-Guided Diffusion Models with DDPM Sampling

Nizar EL GHAZAL, Hatim MRABET

March 2025

Abstract

This report presents our implementation and adaptation of the Pseudoinverse-Guided Diffusion Models (PIGDM) framework for inverse problems. Specifically, we explore modifications where we replace the Denoising Diffusion Implicit Model (DDIM) with the Denoising Diffusion Probabilistic Model (DDPM), adapt the algorithm accordingly, and apply these methods to super-resolution and colorization tasks. Additionally, we investigate using DDIM sampling techniques on a DDPM-trained model, following the exact procedure outlined in the original IIGDM paper. The effectiveness of these approaches is evaluated, and theoretical differences between DDPM and DDIM sampling are analyzed.

1 Introduction

Diffusion models have emerged as powerful generative models, excelling in solving various inverse problems. The *Pseudoinverse-Guided Diffusion Models* (PIGDM) framework proposes an innovative way to leverage problem-agnostic diffusion models to achieve state-of-the-art performance in image restoration tasks, including super-resolution, inpainting... The original method primarily relies on the DDIM approach for efficient sampling. In our work, we explore the impact of using the DDPM model instead, adapting the IIGDM algorithm to the fully stochastic DDPM setting.

Our modifications include:

- Replacing DDIM with DDPM and adapting the sampling algorithm.
- Implementing DDIM sampling on a DDPM-trained model.
- Applying these methods to two inverse problems: super-resolution and colorization.

This project builds upon concepts explored in our coursework, particularly in generative models, inverse problems, and diffusion models. The knowledge gained from these courses has provided the theoretical foundation necessary to understand and implement diffusion-based image restoration techniques. Our work directly applies these concepts in adapting IIGDM to different sampling methods and evaluating their effectiveness in solving inverse problems.

The rest of this report is structured as follows. In Section 2, we provide a theoretical overview of diffusion models and highlight the differences between DDIM and DDPM sampling. Section 3 presents the details of our methodology, including the adaptations made for DDPM-based IIGDM. Section 4 describes our experimental setup, followed by results and analysis in Section 5. Finally, we conclude with insights and possible future directions in Section 6.

2 Theoretical Overview of Diffusion Models

Diffusion models are a class of generative models that iteratively transform noise into structured data through a sequence of denoising steps. They are based on the principle of a forward process

that gradually adds noise to data, followed by a learned reverse process that reconstructs the original data distribution.

A diffusion model consists of:

- **Forward Process (Noise Addition):** A data sample x_0 is progressively corrupted by Gaussian noise over discrete time steps $t = 1, \dots, T$, following a variance schedule β_t , such that:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I). \quad (1)$$

- **Reverse Process (Denoising):** A neural network is trained to estimate the score function (gradient of log-likelihood) to iteratively reverse the noise and recover the original sample.

The effectiveness of diffusion models in image synthesis, restoration, and inverse problems stems from their ability to model complex data distributions efficiently.

2.1 Differences Between DDIM and DDPM Sampling

Denoising Diffusion Probabilistic Models (DDPM) and Denoising Diffusion Implicit Models (DDIM) are two sampling techniques used in diffusion models. While they share the same forward process, they differ in the way they perform reverse sampling:

- **DDPM (Fully Stochastic Sampling):** The reverse process follows a probabilistic approach, where each step involves sampling from a learned noise distribution. This makes DDPM robust but computationally expensive as it requires a large number of sampling steps.
- **DDIM (Deterministic or Semi-Stochastic Sampling):** DDIM modifies the sampling equations to allow for deterministic trajectory sampling, meaning it can generate high-quality samples with fewer steps compared to DDPM. DDIM reduces the stochastic nature of sampling, which results in faster inference.
- **Trade-offs:** DDPM produces diverse samples due to its stochasticity but is slower. DDIM, on the other hand, sacrifices some diversity in favor of speed and efficiency, making it well-suited for real-time applications.

These differences are crucial when adapting IIGDM to work with DDPM instead of DDIM, as the stochasticity of DDPM may require modifications in guidance strategies and inverse problem formulations.

3 Theoretical Background

3.1 Foundational Differences: DDPM vs DDIM

Both methods build on the same forward process but diverge in reverse process parameterization:

- **DDPM (Stochastic):** The reverse process is Markovian with learned Gaussian transitions:

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_t) \quad (2)$$

where Σ_t is either learned or set to $\sigma_t^2 \mathbf{I}$. Sampling requires full diffusion trajectory computation.

- **DDIM (Deterministic)**: Re-parameterizes the reverse process as non-Markovian by defining:

$$\mathbf{x}_{t-1} = \underbrace{\sqrt{\alpha_{t-1}} \left(\frac{\mathbf{x}_t - \sqrt{1 - \alpha_t} \epsilon_\theta(\mathbf{x}_t, t)}{\sqrt{\alpha_t}} \right)}_{\text{Tweedie denoising}} + \sqrt{1 - \alpha_{t-1} - \sigma_t^2} \epsilon_\theta(\mathbf{x}_t, t) \quad (3)$$

where σ_t controls stochasticity ($\sigma_t = 0$ makes it deterministic).

3.2 Algorithmic Implications for IIGDM

The original IIGDM algorithm leverages DDIM's properties through:

$$\mathbf{x}_s = f_{\text{DDIM}}(\mathbf{x}_t) + r_t^2 \nabla_{\mathbf{x}_t} \log p_t(\mathbf{y}|\mathbf{x}_t) \quad (4)$$

Adapting this to DDPM requires three key modifications:

3.2.1 Stochastic Guidance Injection

DDIM's deterministic nature allows direct gradient updates. For DDPM, we must account for the noise term ϵ in the reverse step:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta \right) + \sigma_t \epsilon + r_t^2 \nabla_{\mathbf{x}_t} \log p_t(\mathbf{y}|\mathbf{x}_t) \quad (5)$$

We know that :

$$r_t^2 = \mathbb{V}[x_0|x_t]$$

We have as well, in DDPM :

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta$$

By rewriting this latter formula, we obtain :

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta}{\bar{\alpha}_t}$$

Which implies that :

$$r_t^2 = \frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t}$$

3.2.2 DDPM Adapted Algorithm

Below you will find the pseudocode of the IIGDM algorithm adapted for DDPM sampling method :

Algorithm 1: IIGDM for DDPM

Input:

- Measurement $\mathbf{y}$, measurement model $h(\mathbf{x})$ (noiseless) or $\mathbf{H}, \sigma_{\mathbf{y}}$ (noisy)
- Pretrained DDPM $\epsilon_{\theta}(\mathbf{x}_t, t)$
- Timesteps $\{t_i\}_{i=0}^N$ (descending order, $t_0 = 0, t_N = T$)
- Guidance weight $\eta \in [0, 1]$, number of steps N

Output: Reconstructed sample $\mathbf{x}_0$ **Initialize:**

- Precompute $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s) \forall t$
- If noiseless: $\sigma_{\mathbf{y}} \leftarrow 0$
- Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$

Sampling Loop:**for** $t = T$ **to** 1 **do**

1. Compute $\bar{\alpha}_t, \beta_t$, and r_t :

$$r_t = \sqrt{\frac{1 - \bar{\alpha}_t}{\bar{\alpha}_t}}$$

2. Denoise $\mathbf{x}_t$ to estimate $\hat{\mathbf{x}}_t$:

$$\hat{\mathbf{x}}_t = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}}$$

3. Compute guidance term $\mathbf{g}_t$:

if $\sigma_{\mathbf{y}} = 0$ (*noiseless*) **then**

$$\mathbf{g}_t = \frac{1}{r_t^2} \left(h^{\dagger}(\mathbf{y}) - h^{\dagger}(h(\hat{\mathbf{x}}_t)) \right)^{\top} \frac{\partial \hat{\mathbf{x}}_t}{\partial \mathbf{x}_t}$$

end**else**

$$\mathbf{g}_t = \left((\mathbf{y} - \mathbf{H}\hat{\mathbf{x}}_t)^{\top} \left(\mathbf{H}\mathbf{H}^{\top} + \frac{\sigma_{\mathbf{y}}^2}{r_t^2} \mathbf{I} \right)^{-1} \mathbf{H} \frac{\partial \hat{\mathbf{x}}_t}{\partial \mathbf{x}_t} \right)^{\top}$$

end

4. Update $\mathbf{x}_{t-1}$:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \eta \sqrt{\beta_t} \mathbf{g}_t + \sqrt{\beta_t} \epsilon$$

where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$

end**return** $\mathbf{x}_0 \approx \hat{\mathbf{x}}_1$

4 Experiments and Results

We conducted five experiments to compare the performance of DDIM and DDPM sampling methods across various tasks. All the used images are from the FFHQ dataset used in TP6, and each image has a resolution of 256×256 pixels.

Since both DDPM and DDIM share the same forward process, we use the same denoising model for them (the Unet from TP6).

For DDPM, we use up to 1000 steps. We start at time $t = \text{steps}$, apply the forward process to $\mathbf{y}$ to reach that noise level, then denoise step by step down to $t = 0$.

For DDIM, we use up to 500 steps. We start at $t = 500$ and sample by skipping timesteps at uniform intervals, depending on the number of steps.

Practical Implementation for the Noisy Case. In practice, explicitly constructing and inverting the measurement matrix H , or computing the inverse $\left(\mathbf{H}\mathbf{H}^{\top} + \frac{\sigma_{\mathbf{y}}^2}{r_t^2} \mathbf{I} \right)^{-1}$, is infeasible

for high-dimensional data such as images. Instead, we implement the guidance computation for the noisy case using a conjugate gradients (CG) solver. This allows us to efficiently solve the corresponding linear system in operator form, using only function evaluations of $h(\cdot)$ and its pseudoinverse $h^\dagger(\cdot)$, without ever explicitly constructing the full matrix H .

4.1 Gaussian Blurring

For this experiment, the observation is a Gaussian-blurred image with a kernel size of 31 and a standard deviation of 100. We applied both DDPM and DDIM sampling algorithms using 100 steps. It is worth mentioning that we observe that already the pseudo inverse operator is capable of recovering a lot of details from the blurred images, which makes the task easier for both algorithms. We observed that DDPM performed better than DDIM for this task. To quantify this, we computed the mean PSNR between the original and reconstructed images. DDPM achieved a mean PSNR of approximately 27, while DDIM reached 24.62. The results of this experiment are presented in the appendix in Figures 1 and 3. A closer inspection of the reconstructed images reveals that the results are not perfect: DDPM outputs remain slightly noisy, while DDIM tends to produce oversmoothed images. It is likely that tuning the parameters could lead to improved reconstructions.

4.2 Image Inpainting

In this experiment, we applied the algorithm to the task of image inpainting. We first tested it using random masking with DDIM. We used 100 steps for DDIM and 1000 steps for DDPM. The results were visually good, and the images were reasonably well reconstructed; however, the outputs appeared overly smooth. The corresponding results are shown in Figure 5.

We then tested both DDIM and DDPM using square masks with a fixed block size of 60. The outcomes are presented in Figures 6 and 7. In this setting, DDPM outperformed DDIM—particularly noticeable on Image 2, which DDIM failed to reconstruct, while DDPM successfully recovered it.

4.3 Rotation

We also evaluated the performance of both algorithms using a rotation operator. In this experiment, the images were rotated by 45 degrees. We observed that both DDIM and DDPM struggled to reconstruct the image corners accurately. This limitation is clearly illustrated in Figures 8 and 10. We also observed that the PSNR for DDIM and DDPM are roughly the same (respectively 11.71 and 11.85)—which highlights the poor quality of the reconstructed results. Results of PSNR are represented in the figures 9 and 11.

4.4 Super-Resolution

For this experiment, we use a scale factor of 4 to downsample our images to a resolution of 64×64 , and we test our algorithm on this super-resolution task.

We applied the DDPM algorithm using 500 steps and DDIM using 100 steps. We observed that both methods are able to generate solid results, with DDPM only slightly outperforming DDIM, especially considering the difference in inference time. Visually, the main difference was that DDIM outputs exhibited smoother textures. The results can be observed in Figures 12 and 14.

Quantitatively, both methods produced close results in terms of SSIM: DDPM achieved a score of 0.75, while DDIM achieved 0.74. For PSNR, the difference was slightly more noticeable, with DDPM achieving 23.28 dB and DDIM 22.89 dB. The corresponding results can be seen in Figures 13 and 15.

We also experimented with the noisy algorithm for super-resolution using DDIM. For smaller σ_y values (below 0.1), the results remain solid. However, at $\sigma_y = 0.1$, the outputs start to show very slight differences in facial features (results shown in Figure 16). For higher values of σ_y , the algorithm’s outputs begin to diverge more noticeably from the base image. At $\sigma_y = 0.5$, the images start exhibiting artifacts and defects, and with even larger σ_y values, the algorithm fails to produce any meaningful results. This progression is illustrated in Figure 17.

4.5 Colorization

In this experiment, we applied the algorithm to the task of image colorization, using both DDIM and DDPM sampling strategies. The grayscale images served as inputs, and we tested DDIM with 100 sampling steps and DDPM with 1000 steps.

The visual results are shown in Figures 18 and 20. Both methods produced plausible colorized outputs, but notable differences can be observed. DDIM tends to generate images that appear desaturated, with colors that are often muted and close to grayscale. In contrast, DDPM produces more vibrant and realistic colorizations, even if they occasionally differ from the exact ground truth colors.

The quantitative evaluation, based on PSNR and SSIM metrics (Figures 19 and 21), shows DDIM achieving a PSNR of 14.28 dB and an SSIM of 0.60, while DDPM achieves a slightly higher PSNR of 14.48 dB, but a lower SSIM of 0.57. These results appear to contradict the visual observations. However, it’s important to note that PSNR and SSIM are not well-suited for evaluating colorization tasks, as they primarily measure pixel-wise similarity and structural alignment rather than perceptual quality. DDIM’s outputs may be closer to the grayscale ground truth in terms of pixel similarity (thus higher SSIM), but they lack the vivid and realistic color representation achieved by DDPM.

5 Conclusion

In this report, we presented our adaptation and implementation of the Pseudoinverse-Guided Diffusion Models (IIGDM) framework, extending it to work with both DDIM and DDPM sampling strategies. Our experiments demonstrate that while **DDIM** offers faster inference with reasonably good performance, **DDPM** provides more diverse and often higher-quality reconstructions, especially in tasks like *colorization* and *inpainting*. However, this comes at the cost of increased computation time due to its stochastic nature.

Furthermore, we explored the application of IIGDM to various inverse problems, including *super-resolution*, *colorization*, *inpainting*, *blurring*, and *rotation*. Our analysis highlights the strengths and limitations of each sampling method across different tasks and noise levels. In particular, we found that strong guidance in noiseless scenarios is crucial for faithful reconstructions, while noisy settings require careful tuning of guidance weights to balance fidelity and diversity.

Overall, our work demonstrates the flexibility and effectiveness of the IIGDM framework when adapted to different diffusion models and tasks. Future work could explore integrating text-conditional models or training diffusion models tailored specifically for inverse problems to further enhance performance.

A Appendix

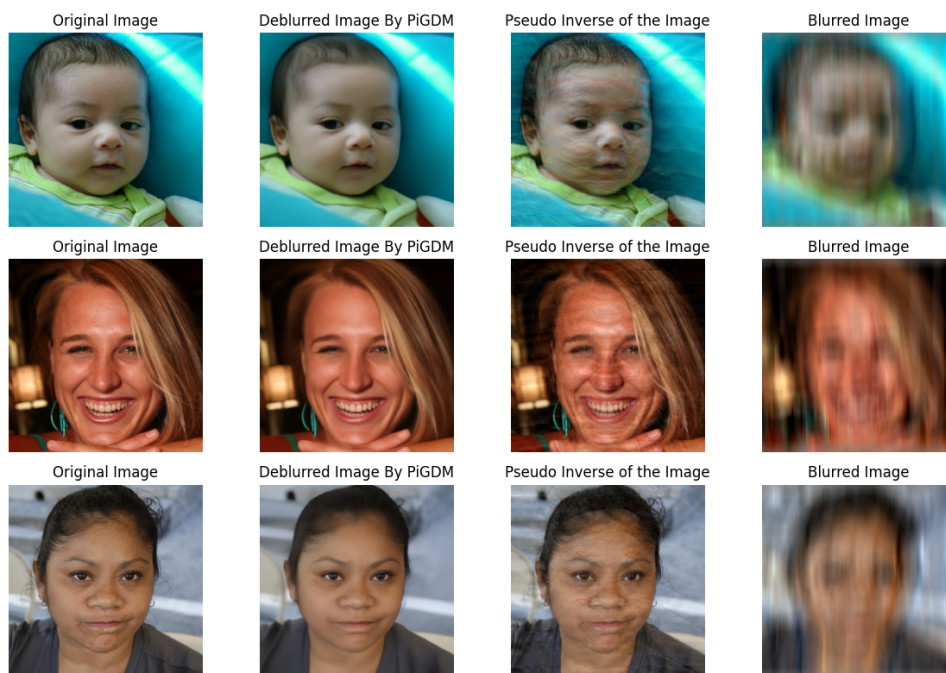


Figure 1: Gaussian Deblurring Results for DDIM Algorithm

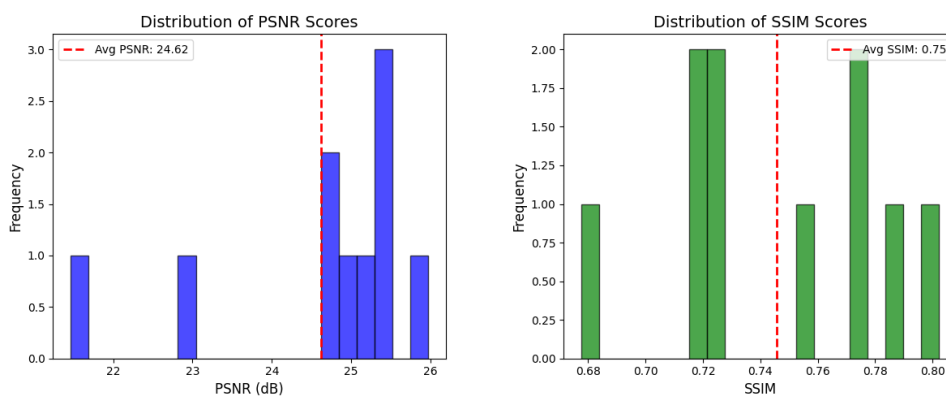


Figure 2: Gaussian Deblurring PSNR and SSIM Results for DDIM Algorithm

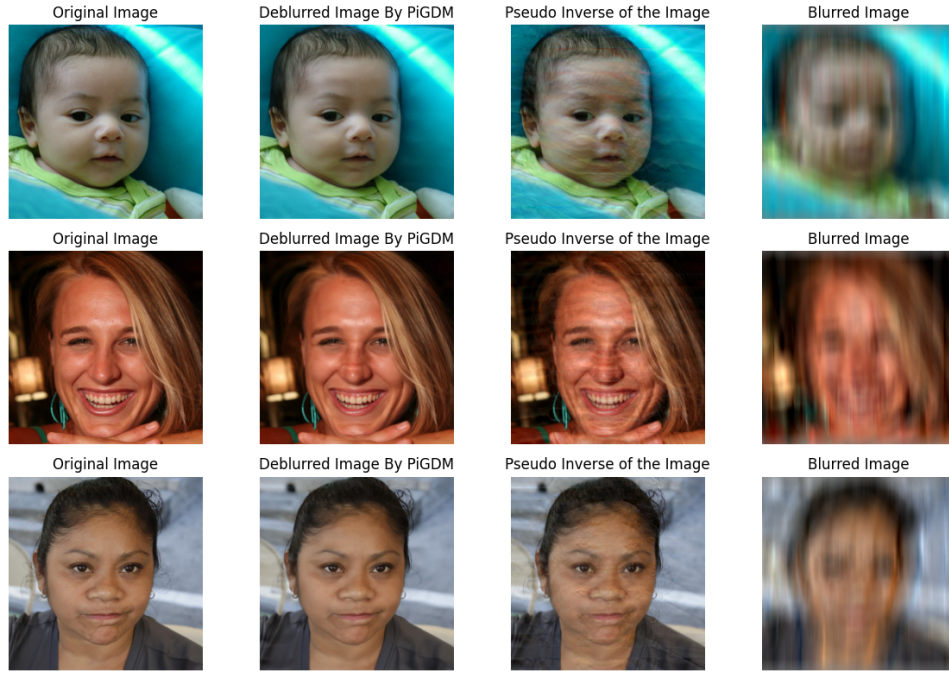


Figure 3: Gaussian Deblurring Results for DDPM Algorithm

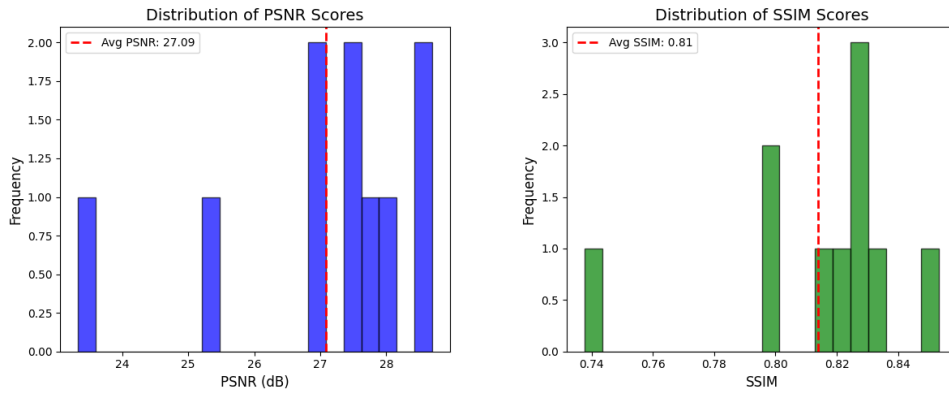


Figure 4: Gaussian Deblurring PSNR and SSIM Results for DDPM Algorithm



Figure 5: Image Inpainting with Random Masking Results for DDIM Algorithm



Figure 6: Image Inpainting with Square Mask Results for DDIM Algorithm



Figure 7: Image Inpainting with Square Mask Results for DDPM Algorithm



Figure 8: Image Rotation Results for DDIM Algorithm

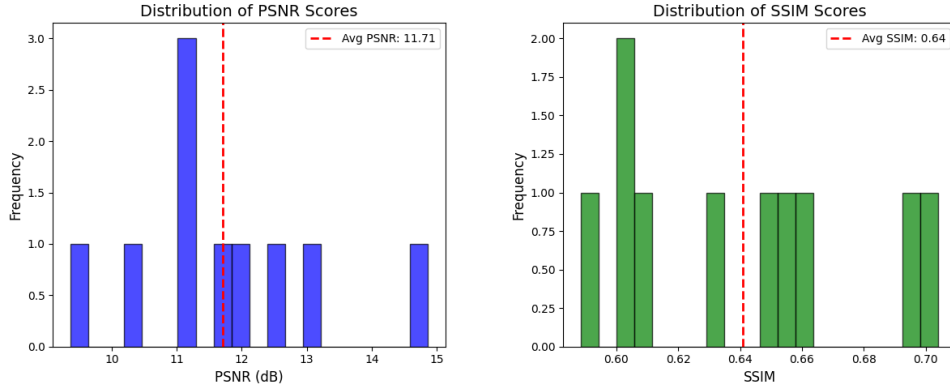


Figure 9: Rotation PSNR and SSIM Results for DDIM Algorithm



Figure 10: Image Rotation Results for DDPM Algorithm

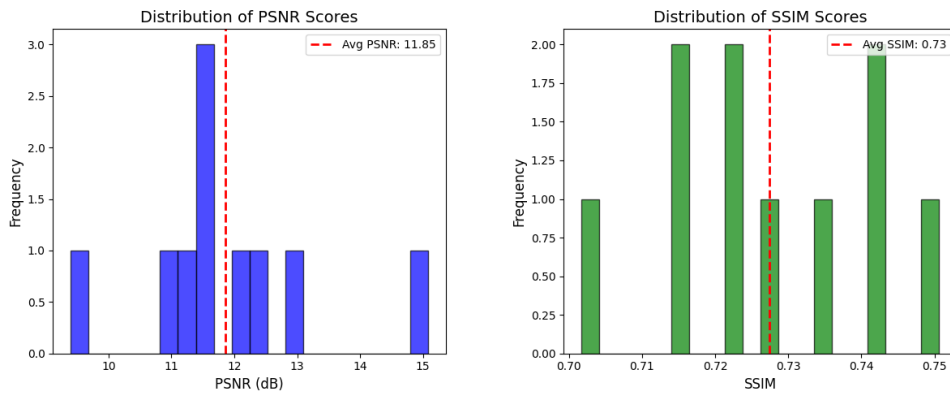


Figure 11: Rotation PSNR and SSIM Results for DDPM Algorithm

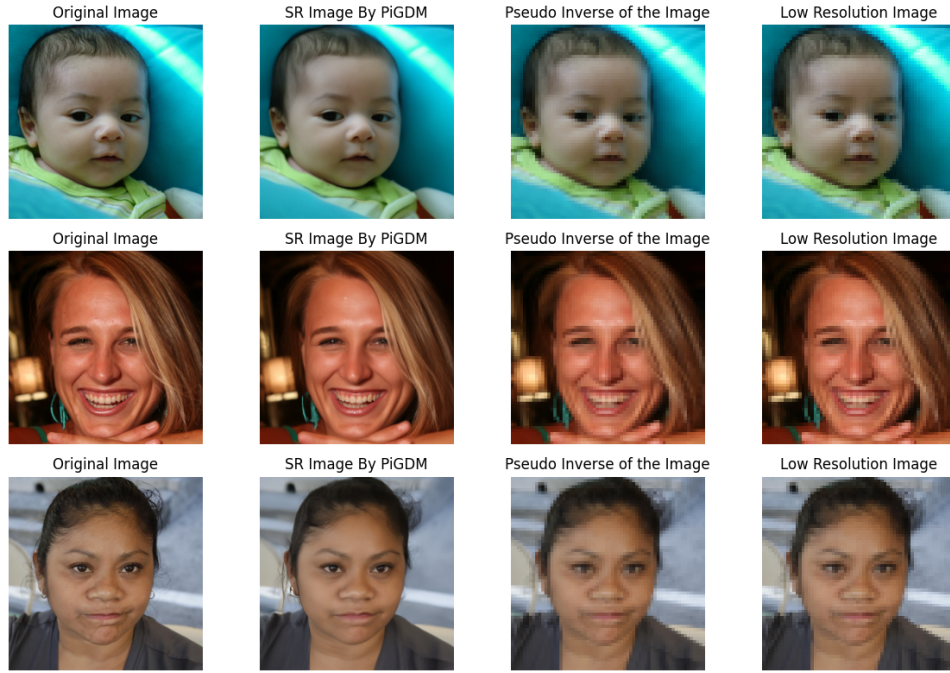


Figure 12: Super Resolution Results for DDIM Algorithm(scale factor = 4)

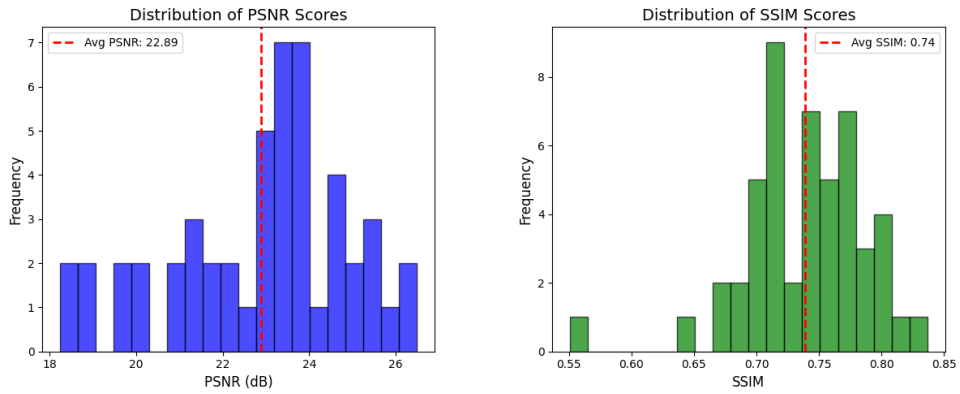


Figure 13: Super Resolution PSNR and SSIM Results for DDIM Algorithm

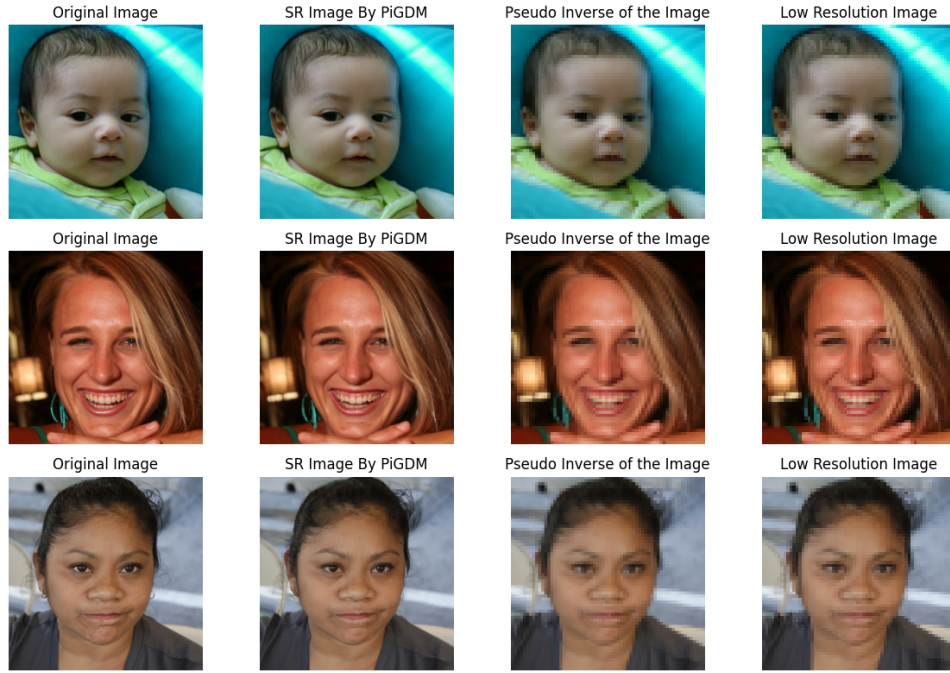


Figure 14: Super Resolution Results for DDPM Algorithm(scale factor = 4)

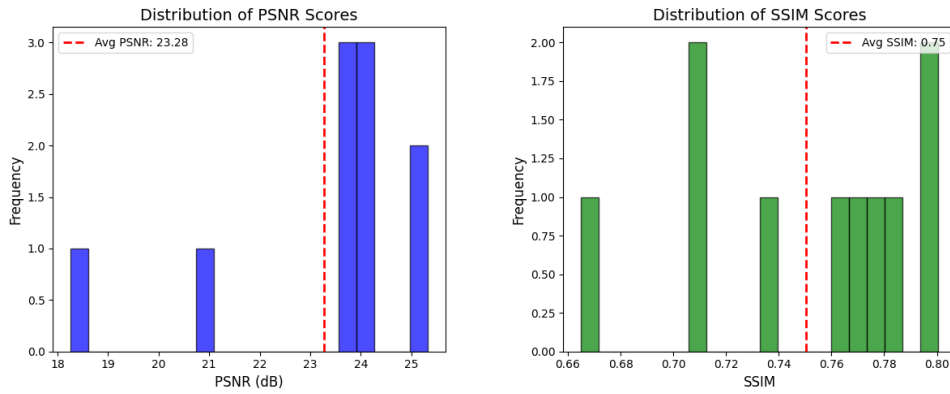


Figure 15: Super Resolution PSNR and SSIM Results for DDPM Algorithm

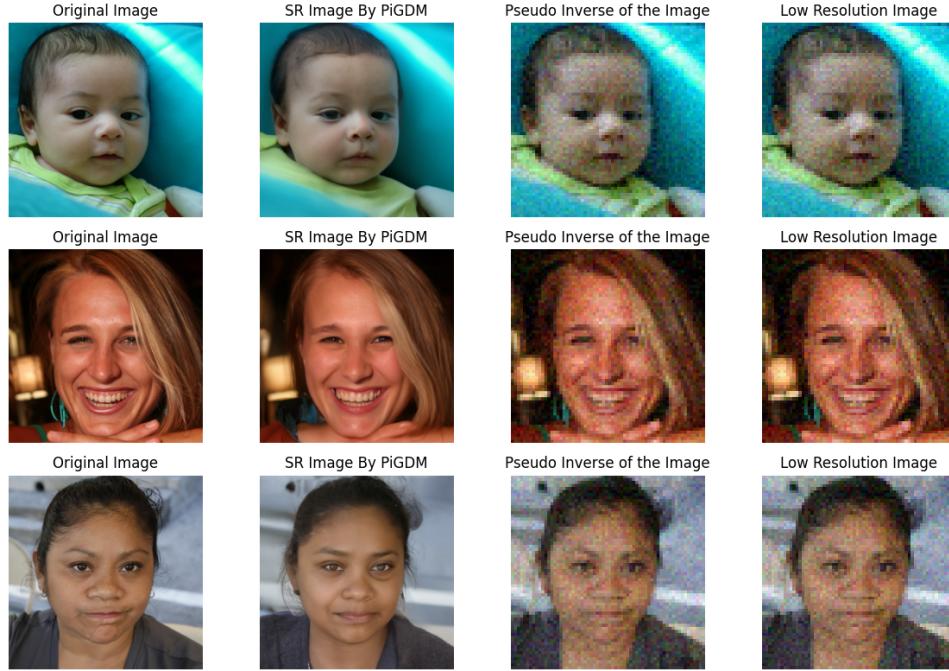


Figure 16: Super Resolution With Noise Results for DDIM Algorithm(scale factor = 4)

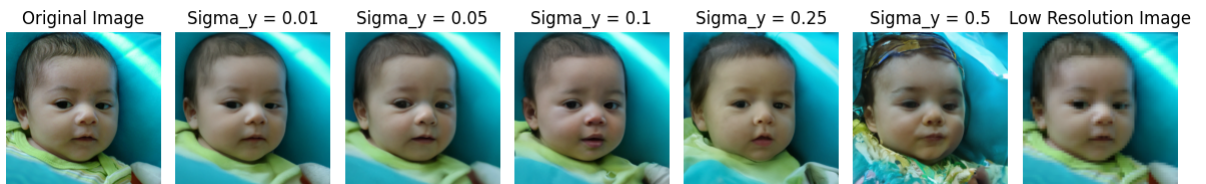


Figure 17: Impact of Noise Standard Deviation on Super Resolution With Noise Results for DDIM Algorithm(scale factor = 4)

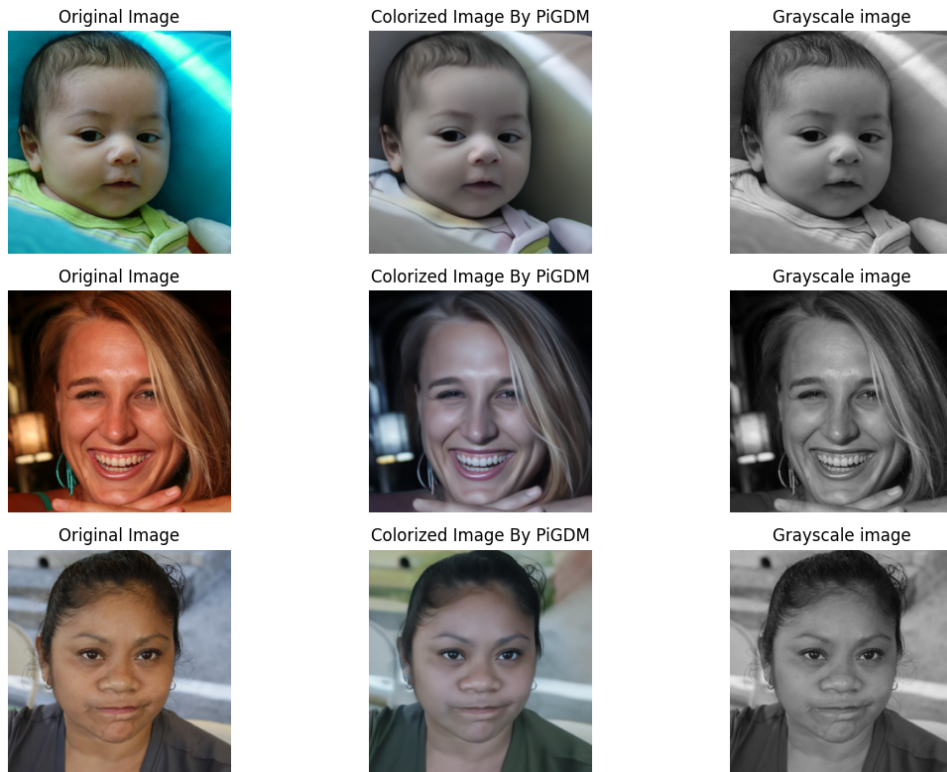


Figure 18: Colorization Results for DDIM Algorithm

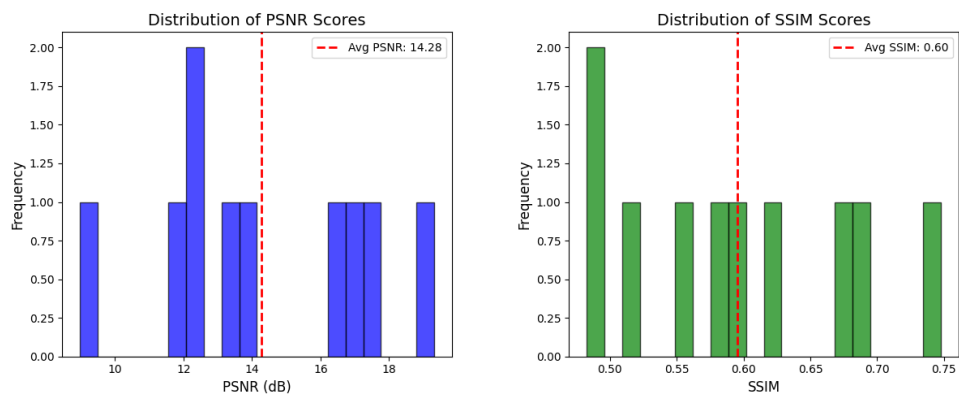


Figure 19: Colorization PSNR and SSIM Results for DDIM Algorithm

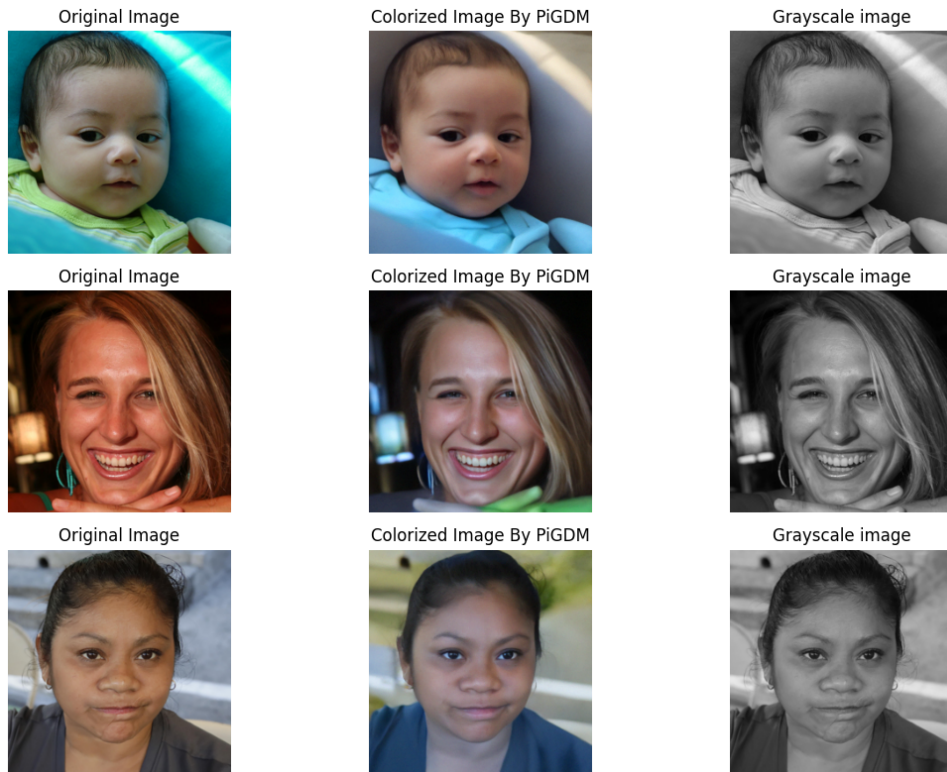


Figure 20: Colorization Results for DDPM Algorithm

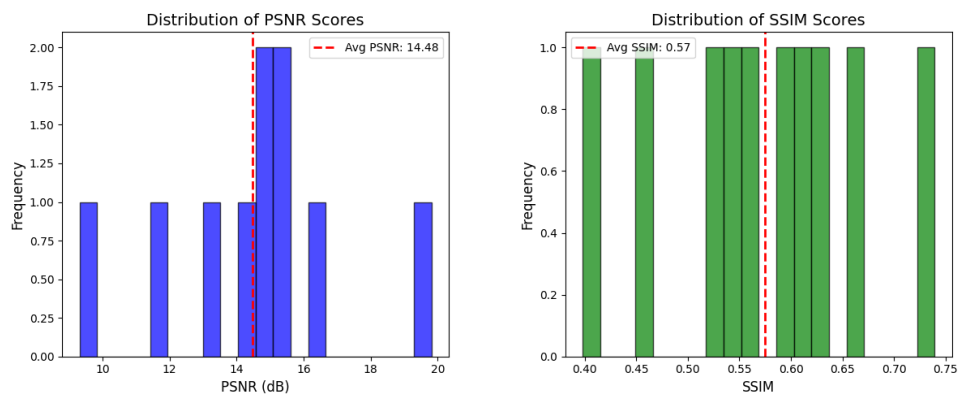


Figure 21: Colorization PSNR and SSIM Results for DDPM Algorithm